

SICNN: Smooth Information Criterion Neural Networks for Sparse Neural Modeling

Aliaksandr Hubin, Andrew McInerney

June 4, 2026

Abstract

Smooth Information Criterion Neural Networks (SICNNs) provide a deterministic, smooth- L_0 approach to sparse neural network training in `torch`. The current implementation optimizes neural weight means directly and adds a Smooth Information Criterion (SIC) penalty to the data-fit loss. Sparsity is induced through the surrogate

$$\phi_\epsilon(w) = \frac{w^2}{w^2 + \epsilon^2},$$

with an epsilon-telescope that progressively sharpens the approximation to an L_0 count. The package supports feed-forward networks with optional input-skip connections for feature selection, convolutional layers with the same smooth weight penalty, sparse-path inference for feed-forward architectures, local gradient explanations, and frequentist uncertainty approximations for those explanations. The method is best viewed as a BIC-calibrated neural pruning framework rather than as a direct application of classical BIC theory to a regular parametric model.

1 Introduction

Modern high-dimensional datasets often require models that are predictive while still revealing which inputs and pathways are being used. Classical neural networks are typically dense and difficult to interpret, while exact subset selection over neural subnetworks is combinatorial. SICNN addresses this by replacing discrete inclusion indicators with a differentiable approximation to the number of nonzero weights. The resulting objective can be optimized with standard gradient-based tools while encouraging weights to collapse toward zero.

The package is related historically to Latent Binary Bayesian Neural Networks (LBBNNs), but the current `train.SICNN()` implementation is deterministic and SIC-based. It does not expose a variational-inclusion training criterion or a KL-divergence objective. Instead, it stores trainable weight means in `SICNN.Linear` and `SICNN.Conv2d` modules, applies the smooth L_0 penalty to those weight tensors, and then derives sparse inference paths by thresholding the trained weights.

2 Current Implementation

2.1 Network Classes

The main feed-forward model is `SICNN_Net`. For layer sizes

$$(p, h_1, \dots, h_L, o),$$

the model creates `SICNN_Linear` layers followed by an output layer. When `input_skip = TRUE`, the original input vector is concatenated into every hidden layer after the first and also into the output layer. These skip connections allow the final sparse graph to represent both direct effects and hidden nonlinear transformations of the input features.

The package also includes `SICNN_Conv2d` and a simple `SICNN_ConvNet` wrapper for image-style classification. Convolutional layers use the same trainable `weight_mean` tensors and the same smooth L_0 penalty. At present, active-path extraction is implemented for feed-forward `SICNN_Net` models; the convolutional wrapper supports sparse forward passes and SIC weight counts, but its active-path density methods return NA and its path-computation methods are stubs.

2.2 The Smooth Information Criterion Penalty

Following O'Neill and Burke (2023), the implementation uses the smooth surrogate

$$\phi_\epsilon(w_k) = \frac{w_k^2}{w_k^2 + \epsilon^2}, \quad (1)$$

which converges pointwise to $\mathbb{I}(w_k \neq 0)$ as $\epsilon \rightarrow 0$. The smooth parameter count is

$$k_\epsilon(W) = \sum_{k=1}^K \phi_\epsilon(w_k), \quad (2)$$

where the sum currently ranges over the trainable weight matrices/tensors in the hidden and output layers. Bias parameters can be present in the model, but they are not included in `smooth_param_count()`, `sic_density()`, or `sic_weight_counts()`.

The default penalty coefficient is

$$\lambda = \log(n_{\text{train}}), \quad (3)$$

although users may provide any positive scalar through the `penalty` argument. Thus the structural term in training is

$$\lambda k_\epsilon(W). \quad (4)$$

2.3 Training Objective

The actual training objective used in `train_SICNN()` is

$$\mathcal{L}_{\text{train}}(W; \epsilon) = \mathcal{L}_{\text{data, scaled}}(W) + \lambda k_\epsilon(W). \quad (5)$$

For regression, `torch::nn_mse_loss(reduction="sum")` is first computed on the mini-batch, and the implementation uses a profile-Gaussian BIC-style scaling

$$\mathcal{L}_{\text{data,scaled}} = n_{\text{train}} \log \left(\frac{\text{RSS}_{\text{batch}}}{m} + 10^{-12} \right), \quad (6)$$

where m is the mini-batch size. For binary and multiclass classification, the implementation uses summed BCE or NLL loss and rescales it to the nominal training-sample size:

$$\mathcal{L}_{\text{data,scaled}} = \frac{n_{\text{train}}}{m} \mathcal{L}_{\text{batch}}. \quad (7)$$

This classification objective is on the negative-log-likelihood scale used by `torch`; if one wants the literal classical $-2\ell + \log(n)k$ scale, the factor of two can be absorbed into the penalty calibration.

The gradient of the smooth penalty is

$$\frac{\partial \phi_\epsilon}{\partial w} = \frac{2w\epsilon^2}{(w^2 + \epsilon^2)^2}. \quad (8)$$

This gradient is largest for weights near the current epsilon scale, giving the optimizer a continuous route for shrinking small coefficients while leaving large coefficients relatively unaffected.

2.4 Epsilon Telescope and Restarts

Training uses an exponential epsilon schedule

$$\epsilon_t = \epsilon_1 \left(\frac{\epsilon_T}{\epsilon_1} \right)^{t/(T-1)}, \quad t = 0, \dots, T-1, \quad (9)$$

with epoch-to-epsilon indexing controlled by `steps.T`. Defaults are `epsilon_1 = 10`, `epsilon_T = 1e-5`, and `steps_T = 100`, although the experiments commonly use `epsilon_1 = 1`.

The `restarts` argument performs multiple deterministic restarts from different random sparse initializations. For restart r , a Bernoulli mask with probability selected from an evenly spaced grid between 0.01 and 0.99 is applied after resetting the layer parameters. The final restart is selected by the mean reported training loss at the end of that run. This is a practical heuristic for exploring local minima, not a guarantee of global optimization.

2.5 Sparse Inference and Active Paths

After training, a weight is considered active if either

$$\phi_{\epsilon_T}(w) > \tau$$

when `sic_threshold_type = "phi"`, or

$$|w| > \tau$$

when `sic_threshold_type = "abs"`. The default is a ϕ -threshold with `sic_threshold = 0.5`.

For feed-forward models, `compute_paths()` and `compute_paths_input_skip()` construct binary threshold matrices and then remove edges that are not on an active route from input to output. Sparse prediction uses the resulting `alpha_active_path` masks by multiplying them into the trained weights during the forward pass. This gives deterministic sparse inference for `SICNN_Net`. For convolutional models, sparse forward passes use each layer’s current `alpha_active_path`; however, the current convolutional wrapper does not compute graph-level active paths.

3 Relationship to BIC and Degrees of Freedom

The SIC objective converges formally to an L_0 -penalized likelihood as $\epsilon \rightarrow 0$:

$$\lim_{\epsilon \rightarrow 0} \left\{ -2\ell(W) + \lambda \sum_k \phi_\epsilon(w_k) \right\} = -2\ell(W) + \lambda \|W\|_0. \quad (10)$$

With $\lambda = \log(n)$ this resembles BIC when the candidate model is a regular parametric model with a fixed, identifiable parameter dimension.

For SICNNs, this connection should be interpreted as a calibration principle rather than a complete consistency theorem. Neural networks are generally non-regular models: hidden units can be permuted, weights can be redundant, and different sparse subnetworks may represent the same function. In addition, the optimizer searches adaptively over many sparse graphs. The active-weight count is therefore not identical to classical effective degrees of freedom. For a fixed sparse neural architecture, the functional degrees of freedom may be lower than the raw number of active weights because of non-identifiability and redundant parameterizations. For the full selection procedure, the effective model-search complexity may be larger than the final active-weight count because the selected graph was not fixed in advance.

Consequently, `penalty = log(n_train)` is a useful BIC-like default, but it is not universally optimal. Stronger penalties such as $2 \log(n)$, $5 \log(n)$, or larger multipliers are often appropriate when the target is structural recovery, when n is small, or when the architecture is wide relative to the sample size.

4 Experimental Results

The repository contains tutorial-style examples, a small saved linear simulation study, convolutional smoke tests, and UCI benchmark scripts. The results below describe the current code and artifacts rather than an idealized theoretical implementation.

4.1 Linear Simulation: BIC Penalty Versus Cross-Validated Penalty

The saved object `rj_experiments/linear_sim_results.rds` contains 30 runs from a synthetic sparse linear regression study. Each run used $n = 1000$, $p = 15$, three true nonzero coefficients, SNR levels 3, 5, and 10, and five repetitions per SNR. The architecture was $(15, 5, 5, 1)$ with input-skip connections. The BIC method used the default $\lambda = \log(n_{\text{train}})$, while the CV method selected λ from 10 log-spaced candidates between 1 and 100 using a held-out validation fold.

Method	SNR	Test MSE	Coef. Error	FPR	Active Weights
BIC	3	0.2486	0.0229	0.0000	7.0
BIC	5	0.1522	0.0193	0.0000	8.0
BIC	10	0.0826	0.0261	0.0333	12.2
CV	3	0.2584	0.0350	0.0833	10.6
CV	5	0.1603	0.0303	0.1000	12.4
CV	10	0.0791	0.0229	0.1000	18.8

Table 1: Mean results from `linear_sim_results.rds`. Both methods achieved mean TPR equal to 1.0 at every SNR. BIC gave similar prediction error with lower false-positive rates and fewer active weights.

In this favorable sparse-linear setting, the BIC-calibrated penalty was conservative and effective: it recovered all true variables in every run and selected fewer false positives than the validation-selected penalty, with similar prediction error.

4.2 Tutorial 1: Sparse Linear Regression

The tutorial `rj_experiments/Tutorial_1_linear_SIC.R` simulates $n = 1000$ observations with $p = 15$ covariates and three active linear effects. It trains an input-skip $(15, 5, 5, 1)$ regression model with `epsilon_1 = 1`, `epsilon_T = 1e-5`, `steps_T = 200`, and a strong explicit penalty `penalty = 80`. This tutorial is therefore an example of aggressive structural pruning rather than default BIC. The associated local explanation figure illustrates how the sparse input-skip model can represent direct linear effects.

4.3 Tutorial 2: Nonlinear Binary Classification

The nonlinear tutorial uses $n = 1000$, $p = 15$, and a response generated from logarithmic, cosine, interaction, and quadratic terms involving six active inputs. The response is dichotomized at the median and fit with a $(15, 5, 5, 1)$ input-skip binary classifier. This script uses the default SIC penalty, $\lambda = \log(n_{\text{train}})$, with `epsilon_1 = 1`, `epsilon_T = 1e-5`, and `steps_T = 200`. The global plot shows the thresholded feed-forward active paths found by the model.

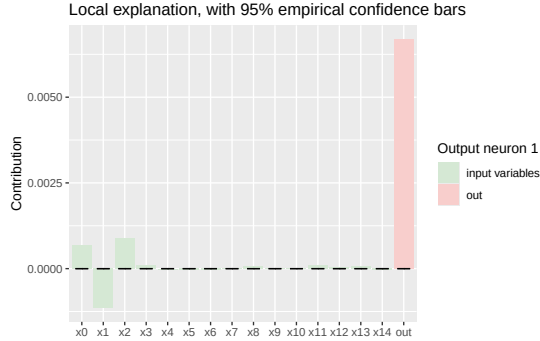


Figure 1: Local explanation pathways for a single prediction in the linear regression tutorial. The explanation is computed as the input gradient of the sparse feed-forward model.

4.4 Tutorial 3: Gallstone Classification

The gallstone tutorial uses the packaged `GallstoneDataset`, trains a $(40, 3, 3, 1)$ input-skip binary classifier, and evaluates sparse prediction on a held-out split. The script uses the default penalty $\lambda = \log(n_{\text{train}})$, although `n_train` is set to the full dataset size in the script rather than the size of the training split. The example demonstrates the intended workflow for tabular clinical classification: standardize inputs, train with SIC, validate both dense and sparse predictions, plot the global sparse graph, and compute local explanations.

4.5 Convolutional KMNIST Tutorial

The KMNIST tutorial builds a two-convolution/two-dense classifier using `SICNN_Conv2d` and `SICNN_Linear`. It trains for one epoch by default and demonstrates that the SIC training loop, smooth parameter counts, sparse forward calls, and plotting method can be applied to the convolutional wrapper. This example should be read as an implementation and scalability demonstration. The current code does not compute active convolutional paths and does not establish exact spatial feature selection.

4.6 Benchmark on UCI Datasets

The UCI benchmark script trains standardized input-skip networks of the form

$$(p, 100, 50, 20, \text{out})$$

and compares sparse SICNN predictions against GLM or multinomial/logistic baselines and random forests. Unlike the default BIC setting, the script uses dataset-specific penalty multipliers $\lambda = c \log(n_{\text{train}})$, with multipliers ranging from 0.005 to 1.0 in the current file. The table below records the values in `uci_experiments/final_uci_benchmarks.log`.

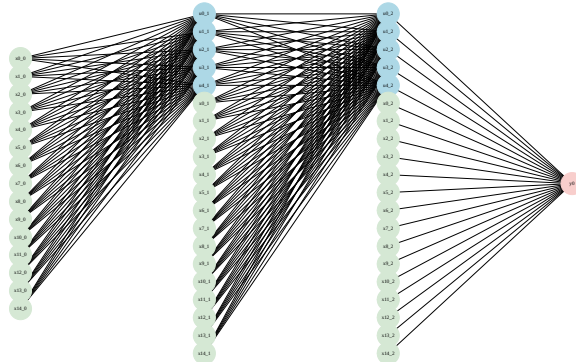


Figure 2: Global sparse structure for the nonlinear classification tutorial. Edges are retained when they are active under the SIC threshold and lie on an input-output path.

5 Practical Tuning Guidance

Penalty coefficient. The default $\lambda = \log(n_{\text{train}})$ is a useful starting point. In sparse linear simulations it produced lower false-positive rates than validation-selected penalties. In noisier, smaller, or wider settings, stronger penalties are often needed for structural recovery. In the current examples, explicit penalties range from BIC-scale values to much stronger values such as `penalty = 80`.

Epsilon schedule. The epsilon telescope controls when the surrogate starts behaving like a hard L_0 count. If ϵ decreases too quickly, weights can be pushed into regions with nearly zero useful gradient. A practical default is to let `steps_T` span much of training, commonly 60–80% of the epochs in larger benchmark scripts.

Thresholding. The default threshold $\tau = 0.5$ on $\phi_{\epsilon_T}(w)$ corresponds to retaining weights whose magnitude exceeds the final epsilon scale. Raising the threshold gives a stricter sparse graph; lowering it can recover weak but useful edges. The alternative `threshold_type = "abs"` uses an absolute weight cutoff.

Learning rate and scheduling. Because the method depends on separating useful weights from weights that should shrink toward zero, very small learning rates can lead to poor structural separation. The examples use Adam with learning rates between 0.001 and 0.01 and often use `scheduler = "step"`.

Biases and parameter counts. Current SIC counts include weights but not biases. If bias terms are important for a scientific degrees-of-freedom calculation, this should be reported separately or handled by fitting with `bias =`

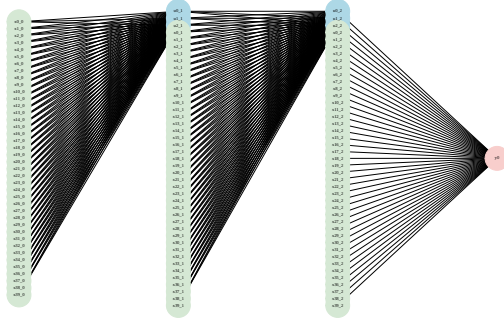


Figure 3: Sparse feed-forward pathways for the gallstone classification tutorial. The graph reflects thresholded active paths in an input-skip **SICNN_Net**.

FALSE.

6 Local Explanations and Uncertainty

For input-skip feed-forward models, local explanations are computed by temporarily using the raw output and differentiating it with respect to the input:

$$g_j(x) = \frac{\partial \mu(W, x)}{\partial x_j}. \quad (11)$$

Sparse explanations use the active-path masked model. The `coef()` method aggregates these gradients over one or more observations and can optionally attach uncertainty estimates.

The uncertainty code approximates parameter covariance using a sandwich-style estimator for penalized models. Let I_0 denote an empirical Fisher approximation from per-sample gradients of the data loss, and let H denote a curvature approximation that adds the nonnegative part of the smooth penalty Hessian,

$$\frac{\partial^2 \phi_\epsilon}{\partial w^2} = \frac{2\epsilon^2(\epsilon^2 - 3w^2)}{(w^2 + \epsilon^2)^3}. \quad (12)$$

The implementation clamps this penalty curvature below at zero before forming diagonal or block-diagonal covariance approximations. For diagonal covariance, the resulting form is

$$\widehat{\text{Var}}(w_j) \approx \frac{I_{0,jj}}{H_{jj}^2}. \quad (13)$$

Dataset	Type	SICNN	GLM	RF	Sparsity
Pima	Classification (Acc)	0.779	0.773	0.773	99.9%
Sonar	Classification (Acc)	0.690	0.690	0.381	100.0%
Breast Cancer	Classification (Acc)	0.956	1.000	0.993	99.9%
Wine	Multiclass (Acc)	0.972	1.000	0.972	99.8%
Airfoil	Regression (R2)	0.570	0.532	0.504	99.9%
Yacht	Regression (R2)	0.970	0.969	0.912	100.0%
Concrete	Regression (R2)	0.419	0.486	0.280	100.0%
Abalone	Regression (R2)	0.472	0.450	0.513	99.9%
Boston	Regression (R2)	0.381	-0.220	0.351	99.9%

Table 2: Performance metrics from the current UCI benchmark log. Sparsity denotes the percentage of penalized weight parameters removed by the final SIC threshold. For regression, R^2 values are reported. Yacht hydrodynamics used a log-transformed target.

For block-diagonal covariance, each parameter tensor is treated as a separate block and inverted independently.

The variance of a local explanation is then approximated by the Delta method:

$$\text{Var}(g(W, x)) \approx J_g \Sigma_W J_g^T, \quad (14)$$

where J_g is computed by automatic differentiation. This provides a scalable post-fit uncertainty heuristic for local explanations. It should not be interpreted as full post-selection inference for the graph-selection procedure.

7 Known Limitations

- The BIC connection is approximate for neural networks because the selected sparse graph is adaptive and the parameterization is generally non-regular.
- The smooth parameter count currently includes weight tensors but excludes biases.
- Feed-forward active-path extraction is implemented; convolutional active-path extraction is not yet implemented.
- Multi-restart training is a local-search heuristic and does not guarantee a global SIC optimum.
- Some legacy documentation strings in the code still refer to variational inference or posterior draws, but current `train_SICNN()` and `predict()` behavior is deterministic SIC training and deterministic forward evaluation.